

Государственный Университет Молдовы
Факультет Математики и Информатики
Кафедра Информатики

Проектная работа

Тема:

Сайт недвижимости UrbanNest Estate

Выполнили студенты: **Splavski Maxim**
Metelska Daniela

Группа: **I2402**

Проверил преподаватель: **Visnevschi V.**

Кишинэу, 2026

Содержание

1	Теоретическая часть	3
2	Формулировка задачи	3
3	Цель и этапы работы	3
4	Практическая часть	3
5	Краткое описание особенностей реализации	4
6	Функциональные возможности	4
7	Основные страницы приложения	4
8	Сценарии взаимодействия	5
9	Механизм регистрации, входа и ролей	5
10	Структура базы данных	5
11	Формы и валидация	6
12	Безопасность	6
13	Выводы	6
14	Ответы на контрольные вопросы	6
15	Список использованных источников	7

1. Теоретическая часть

Веб-приложение UrbanNest Estate реализовано как традиционный PHP-проект, работающий через Apache и MySQL в среде XAMPP. Серверная часть отвечает за обработку форм, работу с базой данных, проверку пользователей и формирование HTML-страниц.

Архитектурно проект наследует принципы MVC. Модели отвечают за работу с таблицами базы данных, контроллеры обрабатывают запросы, а представления формируют интерфейс. Такой подход делает код понятнее и уменьшает смешивание логики с HTML.

Проектная работа выполнена двумя студентами: Splavski Maxim и Metelska Daniela. Оба участника указаны на титульном листе отчета как авторы работы.

2. Формулировка задачи

Необходимо разработать веб-приложение для просмотра и подбора современной недвижимости. Пользователь должен иметь возможность просматривать каталог, искать объекты, регистрироваться, входить в систему и отправлять заявку на просмотр. Администратор должен управлять объектами, типами недвижимости, заявками и пользователями.

3. Цель и этапы работы

Цель работы - создать завершенный MVP сервиса UrbanNest Estate на PHP и MySQL в рамках общей проектной работы двух студентов.

Основные этапы:

1. проектирование структуры приложения;
2. создание базы данных;
3. реализация регистрации и входа;
4. реализация каталога недвижимости;
5. реализация формы заявки на просмотр;
6. разработка административной панели;
7. оформление интерфейса и документации.

4. Практическая часть

Проект реализован без REST API и без внешних backend-фреймворков. Единой точкой входа является файл `public/index.php`. Он определяет маршрут и вызывает соответствующий контроллер.

Код разделен на директории `app/models`, `app/controllers`, `app/views`, `app/core` и `app/config`. Подключение к базе данных вынесено в отдельный класс `Database`, который создает PDO-соединение.

5. Краткое описание особенностей реализации

Приложение использует модульную MVC-подобную архитектуру. В директории `models` находятся классы для работы с пользователями, объектами недвижимости, типами недвижимости и заявками. Контроллеры принимают запросы, проверяют права доступа и передают данные в представления.

Маршрутизация реализована в файле `public/index.php`. Для Apache добавлен файл `public/.htaccess`, который направляет пользовательские URL в единую точку входа. Основные настройки проекта и базы данных находятся в `app/config/config.php`.

Фотографии объектов недвижимости хранятся локально. Для тестовых данных используются квадратные JPG-файлы размером 900 на 900 пикселей из `public/assets/img/properties`. Если администратор загружает новое изображение, система проверяет, что исходный файл имеет размер не меньше 900 на 900 пикселей, выполняет центральную квадратную обрезку, уменьшает результат до 900 на 900 пикселей и сохраняет готовый JPG-файл в `public/uploads/properties`. В базе данных сохраняется только относительный путь.

6. Функциональные возможности

В приложении реализованы:

- главная страница с hero-блоком, быстрым поиском и избранными объектами;
- каталог недвижимости с поиском, фильтрами и сортировкой;
- детальная страница объекта;
- регистрация, вход и выход;
- личный кабинет пользователя;
- форма заявки на просмотр;
- административная панель;
- управление объектами, типами недвижимости, заявками и пользователями.

7. Основные страницы приложения

Главная страница содержит крупный заголовок, краткое описание сервиса, быстрый поиск и блок объектов недели. Каталог недвижимости выводит данные из базы и поддерживает поиск

по названию, городу или району, фильтр по типу объекта, типу сделки, комнатам, статусу, цене и площади.

Страница объекта показывает изображение, город, район, адрес, цену, площадь, количество комнат, этаж, тип объекта, тип сделки, описание и статус. Форма заявки доступна авторизованным пользователям и содержит объект, имя, телефон, email, удобное время связи и комментариев.

Административная панель содержит dashboard со статистикой, управление объектами недвижимости, типами недвижимости, заявками и пользователями. Администратор может создавать новых администраторов и изменять роли пользователей.

8. Сценарии взаимодействия

Гость открывает сайт, просматривает каталог, фильтрует объекты по типу сделки и бюджету, затем изучает страницу выбранного объекта. Для отправки заявки он регистрируется или входит в аккаунт.

Пользователь выбирает объект недвижимости, указывает контактные данные, удобное время связи и комментариев. После отправки заявка появляется в личном кабинете.

Администратор входит в систему, открывает dashboard, просматривает статистику, добавляет объекты, редактирует типы недвижимости и меняет статусы заявок.

9. Механизм регистрации, входа и ролей

Регистрация выполняется через форму с именем, email, телефоном и паролем. На сервере проверяются длина имени, корректность email, телефон, длина пароля и уникальность email. Пароль сохраняется в таблице `users` только после обработки функцией `password_hash()`.

При входе пользователь вводит email и пароль. Контроллер находит пользователя по email и проверяет пароль функцией `password_verify()`. После успешного входа основные данные пользователя сохраняются в `$_SESSION`. Для доступа к личному кабинету используется проверка авторизации, а для административных страниц дополнительно проверяется роль `admin`.

10. Структура базы данных

База данных содержит четыре основные таблицы:

- `users` - пользователи и роли;
- `property_types` - типы недвижимости;
- `properties` - характеристики объектов недвижимости;
- `property_requests` - заявки пользователей на просмотр.

Таблицы связаны внешними ключами. Объект относится к типу недвижимости, а заявка связана с пользователем и объектом недвижимости.

11. Формы и валидация

В проекте реализованы несколько HTML-форм. Форма регистрации создает нового пользователя. Форма поиска в каталоге позволяет фильтровать объекты по нескольким критериям. Форма заявки содержит объект недвижимости, имя, телефон, email, удобное время связи и комментарий.

Административная форма объекта содержит более пяти полей: название, тип, сделку, город, район, адрес, комнаты, площадь, этаж, общую этажность, цену, статус, изображение и описание. На клиенте используются атрибуты `required`, `minlength`, `type=email`, `type=number` и другие. На сервере контроллеры выполняют дополнительную проверку данных перед сохранением.

12. Безопасность

Пароли сохраняются в базе только в хешированном виде. Для проверки пароля используется функция `password_verify()`. Доступ к личному кабинету защищен PHP-сессиями, а административный раздел дополнительно проверяет роль пользователя.

Все SQL-запросы выполняются через подготовленные запросы PDO. Данные при выводе экранируются функцией `htmlspecialchars()`, чтобы снизить риск XSS.

Загрузка изображений ограничена исходными форматами JPG, PNG и WEBP. Минимальный размер исходной фотографии - 900 на 900 пикселей. После загрузки изображение автоматически приводится к квадратному формату 900 на 900 пикселей и сохраняется в отдельную директорию `public/uploads/properties`, а имя файла генерируется автоматически.

13. Выводы

В ходе совместной работы было создано завершенное PHP + MySQL приложение UrbanNest Estate. Проект демонстрирует работу с базой данных, формами, ролями, сессиями, CRUD-операциями, фильтрацией и адаптивным интерфейсом.

Ссылка на репозиторий добавляется после загрузки проекта на GitHub.

14. Ответы на контрольные вопросы

1. Серверная часть обрабатывает запросы, работает с базой данных и формирует страницы.
2. База данных нужна для хранения пользователей, объектов недвижимости, типов и заявок.
3. Аутентификация проверяет личность пользователя.

4. Авторизация определяет доступные пользователю действия.
5. Пароли нельзя хранить открыто из-за риска утечки.
6. PHP-сессии сохраняют состояние входа пользователя.
7. SQL-инъекция - внедрение вредоносного SQL через ввод пользователя.
8. Prepared statements отделяют SQL-код от данных.
9. Валидация проверяет корректность введенных данных.
10. Админ-панель нужна для управления данными проекта.
11. Роль пользователя определяет уровень доступа.
12. Адаптивная верстка делает сайт удобным на разных устройствах.
13. MVC разделяет данные, обработку и представление.
14. Модульная структура легче поддерживается и проверяется.

15. Список использованных источников

- Документация PHP: <https://www.php.net/>
- Документация MySQL: <https://dev.mysql.com/doc/>
- Документация XAMPP: <https://www.apachefriends.org/>
- Wikimedia Commons: фотографии недвижимости для локальных тестовых данных.